

Practical 1: Write a simple client-server application using RPC where the client sends two integers, and the server returns their sum. Compare performance and behaviour of a local function call vs. an RPC call with timing outputs.

Concept

In **Remote Procedure Call (RPC)**, a client calls a function on a remote server as if it were local.

The system handles:

- Network communication
- Serialization/deserialization
- Execution on server

1. System Overview

Client

- Sends two integers (a , b) to server
- Waits for response

Server

- Receives request
- Computes sum
- Returns result

2. Algorithm

Client-Side Algorithm

1. Start client
2. Input integers a and b
3. Record start time
4. Send request (a , b) to server via RPC
5. Wait for response

6. Receive result sum
7. Record end time
8. Display result and time taken

Server-Side Algorithm

1. Start server and listen for requests
2. When request arrives:
 - Extract integers a , b
3. Compute:
$$sum = a + b$$
4. Send result back to client
5. Repeat for next request

Why RPC is Slower

- Network latency
- Data serialization/deserialization
- Context switching
- Communication overhead

Behavior Comparison

Local Call

- Direct execution
- No network
- Immediate response

RPC Call

- Requires connection setup
- May fail due to network issues
- Adds delay

3. Pseudocode

Client (RPC Call)

```
function client():
    input a, b

    start_time = current_time()

    # Remote call
    result = rpc_call("add", a, b)

    end_time = current_time()

    print("Sum =", result)
    print("RPC Time =", end_time - start_time)
```

Server (RPC Service)

```
function add(a, b):
    return a + b

function server():
    register_rpc_method("add", add)
    while True:
        listen_for_requests()
```

Local Function Call (for comparison)

```
function local_add(a, b):
    return a + b

function test_local():
    input a, b

    start_time = current_time()

    result = local_add(a, b)

    end_time = current_time()

    print("Sum =", result)
    print("Local Call Time =", end_time - start_time)
```

4. Example Execution

Input

```
a = 10  
b = 20
```



Local Call Output

```
Sum = 30  
Time = 0.000001 sec
```



RPC Call Output

```
Sum = 30  
Time = 0.02 sec
```



5. Performance Comparison

Feature	Local Function Call	RPC Call
Execution Location	Same process	Remote server
Speed	Very fast	Slower
Overhead	Minimal	High (network + serialization)
Failure Risk	Low	Higher (network issues)
Transparency	Direct	Appears local but remote

Practical 2 : Write a RMI application that allows a client to perform basic arithmetic operations (add, subtract, multiply, divide) on the server.

Concept

Java RMI (Remote Method Invocation)

allows a Java program (client) to invoke methods on a remote object (server JVM) as if they were local.

In this application:

- Client requests arithmetic operations
- Server performs computation
- Result is returned remotely

1. System Architecture

Components

1. **Remote Interface** → declares methods
2. **Remote Object (Server)** → implements interface
3. **RMI Registry** → binds object
4. **Client** → invokes methods

2. Algorithm

Server-Side Algorithm

1. Define remote interface with methods:
 - `add(a, b)`
 - `subtract(a, b)`
 - `multiply(a, b)`

- `divide(a, b)`

2. Implement the interface in server class
3. Create remote object
4. Register object with RMI registry
5. Wait for client requests
6. Execute requested method and return result

Client-Side Algorithm

1. Lookup remote object from registry
2. Take user input (`a`, `b`)
3. Call remote methods
4. Receive results
5. Display output

Key Features

- Platform independent (Java-based)
- Object-oriented RPC mechanism
- Supports distributed applications

Advantages

- Easy remote communication
- Reusability of server logic
- Transparent method invocation

3. Pseudocode

Remote Interface

```
interface Arithmetic extends Remote {  
    int add(int a, int b) throws RemoteException;  
    int subtract(int a, int b) throws RemoteException;  
    int multiply(int a, int b) throws RemoteException;  
    int divide(int a, int b) throws RemoteException;  
}
```

Server

```
class ArithmeticImpl extends UnicastRemoteObject implements Arithmetic {  
  
    ArithmeticImpl() throws RemoteException {}  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int subtract(int a, int b) {  
        return a - b;  
    }  
  
    int multiply(int a, int b) {  
        return a * b;  
    }  
  
    int divide(int a, int b) {  
        return a / b;  
    }  
}  
  
function server():  
    create object of ArithmeticImpl  
    bind object to registry with name "ArithmeticService"  
    print "Server ready"
```

Client

```
function client():  
    lookup "ArithmeticService" from registry  
  
    input a, b  
  
    print "Add =", remote.add(a, b)  
    print "Subtract =", remote.subtract(a, b)  
    print "Multiply =", remote.multiply(a, b)  
    print "Divide =", remote.divide(a, b)
```

4. Example Execution

Input

```
a = 20  
b = 10
```



Output

```
Add = 30  
Subtract = 10  
Multiply = 200  
Divide = 2
```



5. Explanation of Working

1. Server starts and registers remote object
2. Client connects using RMI registry
3. Client calls methods like:

```
remote.add(20, 10)
```



4. Request goes over network
5. Server executes method
6. Result returned to client



Practical 3 : Write Berkeley's Clock Synchronization Algorithm. Provide a detailed example to illustrate how the algorithm synchronizes the clocks of different nodes in the system.

Concept

Berkeley Clock Synchronization

Algorithm is used in distributed systems to **synchronize clocks of multiple nodes** without relying on an external time source (like UTC).

- One node acts as a **master (time server)**, Other nodes are **slaves**
- The master calculates an **average time** and adjusts all clocks accordingly

1. Key Idea

Instead of syncing to real time, the system:

- Collects time from all nodes
- Computes **average time**
- Sends **offset adjustments** to each node

2. Algorithm

Master Node:

1. Periodically send time request to all slave nodes
2. Receive current time from each slave
3. Calculate **time differences (offsets)**:

```
offset = slave_time -  
master_time
```

4. Ignore faulty/outlier clocks (optional)
5. Compute **average offset**:

```
avg_offset = sum(offsets) /  
number_of_nodes
```

6. Send **adjustment value** to each node:

```
adjustment = avg_offset -  
individual_offset
```

Slave Node:

1. Receive request from master
2. Send current time to master
3. Receive adjustment value
4. Update local clock:

```
new_time = current_time +  
adjustment
```

Key Features

- No need for external time source
- Uses **average-based synchronization**
- Handles faulty clocks (by ignoring outliers)

Advantages

- Efficient for internal network synchronization
- Reduces time drift across nodes
- Simple implementation

3. Pseudocode

Master Process

```
function synchronize_clocks(nodes):
    master_time = get_current_time()
    offsets = []

    for node in nodes:
        slave_time = request_time(node)
        offset = slave_time - master_time
        offsets.append(offset)

    avg_offset = sum(offsets) / len(offsets)

    for i in range(len(nodes)):
        adjustment = avg_offset - offsets[i]
        send_adjustment(nodes[i], adjustment)
```

Slave Process

```
function respond_to_master():
    send current_time to master

function adjust_clock(adjustment):
    current_time = current_time + adjustment
```

4. Detailed Example

Initial Times

Node	Time (seconds)
Master	100
S1	102
S2	98
S3	104

Step 1: Calculate Offsets

S1: $102 - 100 = +2$

S2: $98 - 100 = -2$

S3: $104 - 100 = +4$

Step 2: Compute Average Offset

$\text{avg_offset} = (2 + (-2) + 4) / 3$

$= 4 / 3$

≈ 1.33 seconds

Step 3: Calculate Adjustments

S1: $1.33 - 2 = -0.67$

S2: $1.33 - (-2) = +3.33$

S3: $1.33 - 4 = -2.67$

Master adjusts by $+1.33$

Step 4: New Times

Node	Old Time	Adjustment	New Time
Master	100	+1.33	101.33
S1	102	-0.67	101.33
S2	98	+3.33	101.33
S3	104	-2.67	101.33

Final Result

All clocks synchronized to **~101.33 seconds**

Practical 4 : Write the Bully Algorithm for electing a new coordinator in a distributed system. Provide a detailed example to demonstrate how the algorithm works and elects a new coordinator.

Concept

The **Bully Algorithm** is used to **elect a coordinator (leader)** in a distributed system when the current coordinator fails.

- Each process has a **unique ID**
- The process with the **highest ID becomes the coordinator**
- Higher-ID processes can “bully” lower-ID ones

1. Key Idea

- When a process detects coordinator failure:
 - It starts an **election**
- It contacts all **higher-ID processes**
- If none respond → it becomes coordinator
- Otherwise → higher process takes over

2. Algorithm

When a Process Detects Failure

1. Send **ELECTION message** to all processes with higher IDs
2. If **no response**:
 - Declare itself as coordinator
 - Send **COORDINATOR message** to all lower processes

3. If **response received (OK message)**:
 - Wait for higher process to conduct election
4. If no coordinator message is received within timeout:
 - Restart election

When a Process Receives ELECTION

1. Send **OK message** back
2. Start its own election (if not already running)

When a Process Receives COORDINATOR

1. Accept the sender as new coordinator
2. Stop any ongoing election

Key Features

- Highest-ID process always wins
- Multiple elections may occur simultaneously
- Fast recovery

Advantages

- Simple and easy to implement
- Guarantees a unique coordinator
- Works well in small system

3. Pseudocode

Election Initiation

```
function start_election(process_id, processes):  
    higher_processes = [p for p in processes if p > process_id]  
  
    send ELECTION to all higher_processes  
  
    if no OK received:  
        become_coordinator()  
        send COORDINATOR to all lower processes  
    else:  
        wait for COORDINATOR message
```

On Receiving ELECTION

```
function on_receive_election(sender_id):  
    send OK to sender_id  
  
    if not already_in_election:  
        start_election(self_id, processes)
```

On Receiving COORDINATOR

```
function on_receive_coordinator(new_coordinator):  
    coordinator = new_coordinator  
    stop election
```

4. Detailed Example

Processes

P1, P2, P3, P4, P5

(IDs: $1 < 2 < 3 < 4 < 5$)

- Current coordinator = **P5**
- P5 crashes

Step-by-Step Execution

Step 1: Failure Detection

- P2 detects that P5 is down

Step 2: P2 Starts Election

- Sends **ELECTION** → P3, P4, P5

P2 → P3, P4, P5

Step 3: Responses

- P3 and P4 respond with **OK**
- P5 does not respond (crashed)

P3 → OK to P2

P4 → OK to P2

Step 4: Higher Processes Start Election

P3 starts election

- Sends **ELECTION** → P4, P5

P4 starts election

- Sends **ELECTION** → P5

Step 5: Final Election

- P5 is down
- P4 gets no response

Step 6: P4 Becomes Coordinator

P4 → **COORDINATOR** → P1, P2, P3

Step 7: All Processes Update

- New coordinator = **P4**

5. Message Flow Summary

P2 → P3, P4 (ELECTION)

P3, P4 → OK

P3 → P4 (ELECTION)

P4 → (no response from P5)

P4 → **COORDINATOR** to all

Practical 5 : Write the Ring Algorithm for electing a new coordinator in a distributed system. Provide a detailed example to demonstrate how the algorithm works and elects a new coordinator.

Concept

The **Ring Algorithm** is used to elect a **new coordinator** in a distributed system where processes are arranged in a **logical ring**.

- Each process has a **unique ID**
- Messages circulate in a **unidirectional ring**
- The process with the **highest ID** is **elected coordinator**

1. Key Idea

- A process detecting failure starts an **election message**
- Message travels around the ring collecting IDs
- Highest ID is selected as coordinator
- Coordinator announcement circulates to all

2. Algorithm

Election Phase

1. A process detects coordinator failure
2. It creates an **ELECTION message** containing its own ID
3. Sends message to its **next neighbor in the ring**
4. Each process:
 - Adds its ID to the message
 - Forwards it to the next process

Coordinator Selection

5. When message returns to initiator:
 - It finds the **highest ID** in the list
6. That process is selected as **new coordinator**

Coordinator Announcement

7. Initiator sends **COORDINATOR message** with selected ID
8. Message circulates around ring
9. All processes update their coordinator

Key Features

- Uses logical ring structure
- Equal participation of all processes
- No process dominance (unlike Bully Algorithm)

Advantages

- Fair election process
- Lower message complexity than Bully in some cases
- Simple and structured

3. Pseudocode

Election Initiation

```
function start_election(process_id):  
    message = [process_id]  
    send_to_next(message)
```

On Receiving Election Message

```
function on_receive_election(message):  
    if self_id not in message:  
        message.append(self_id)  
  
    if message[0] == self_id:  
        # Message completed full circle  
        coordinator = max(message)  
        send_coordinator_message(coordinator)  
    else:  
        send_to_next(message)
```

Coordinator Announcement

```
function send_coordinator_message(coordinator):  
    send_to_next(["COORDINATOR", coordinator])  
  
function on_receive_coordinator(message):  
    coordinator = message[1]  
  
    if next_process != initiator:  
        send_to_next(message)
```

4. Detailed Example

Processes in Ring

P1 → P2 → P3 → P4 → P5 → P1

IDs: 1, 2, 3, 4, 5

- Current coordinator = **P5**
- P5 crashes

Step-by-Step Execution

Step 1: Failure Detection

- P2 detects that coordinator (P5) is down
-

Step 2: Election Initiated by P2

Message: [2]

P2 → P3

Step 3: Message Circulates

P3 receives → [2,3] → sends to P4

P4 receives → [2,3,4] → sends to P5 (fails) → skips → P1

P1 receives → [2,3,4,1] → sends to P2

Step 4: Message Returns to Initiator

- P2 receives: [2,3,4,1]
 - Highest ID = 4
-

Step 5: Coordinator Announcement

P2 → COORDINATOR(4) → P3 → P4 → P1 → P2

Step 6: All Processes Update

- New coordinator = **P4**
-

5. Message Flow Summary

Election: P2 → P3 → P4 → P1 → P2

Coordinator: P2 → P3 → P4 → P1 → P2

Practical 6 : Write Ricart Agrawala's Algorithm for achieving mutual exclusion in a distributed system. Support your explanation with an example.

Concept

The **Ricart–Agrawala Algorithm** is used to ensure that **only one process enters the Critical Section (CS)** at a time in a distributed system.

- Based on **timestamp ordering (Lamport clocks)**
- No central coordinator required
- Uses **message passing**

1. Key Idea

- A process must get **permission from all other processes** before entering CS
- Requests are ordered using **timestamps**
- Lower timestamp → higher priority

2. Message Types

1. **REQUEST(timestamp, process_id)**
2. **REPLY**

3. Algorithm

When a Process Wants to Enter CS

1. Increment logical clock
2. Send **REQUEST(timestamp, id)** to all other processes

3. Wait until:
 - **REPLY** received from all processes
4. Enter Critical Section

When a Process Receives REQUEST

1. If not interested in CS:
 - Send **REPLY** immediately
2. If in CS:
 - Queue the request (defer reply)
3. If also requesting CS:
 - Compare timestamps:
 - If sender has **lower timestamp** → send **REPLY**
 - Else → defer reply

When Process Exits CS

1. Send **REPLY** to all deferred requests
2. Clear request queue

Key Features

- Fully distributed (no central control)
- Uses **2(N-1) messages per CS entry**
- Ensures **mutual exclusion**

Advantages

- No deadlock
- No starvation (fair ordering)
- Simple logic using timestamp

4. Pseudocode

Process Requesting CS

```
function request_CS():
    clock += 1
    timestamp = clock

    for each process P:
        send REQUEST(timestamp, self_id) to P

    wait until REPLY received from all processes

    enter_CS()
```

On Receiving REQUEST

```
function on_receive_request(ts, sender_id):
    clock = max(clock, ts) + 1

    if not requesting_CS and not in_CS:
        send REPLY to sender_id

    elif in_CS:
        defer request

    elif requesting_CS:
        if (ts < my_timestamp) or
           (ts == my_timestamp and sender_id < self_id):
            send REPLY to sender_id
        else:
            defer request
```

On Releasing CS

```
function release_CS():
    for each deferred request:
        send REPLY
    clear deferred queue
```

5. Detailed Example

Processes

P1, P2, P3

Scenario

- P1 and P2 request CS at the same time
-

Step 1: Requests Generated

P1 → REQUEST(5, P1)

P2 → REQUEST(6, P2)

Step 2: Message Exchange

At P2 (receives P1's request)

- Compare timestamps:

P1 (5) < P2 (6)

- P2 sends REPLY to P1
-

At P1 (receives P2's request)

- P1 has lower timestamp → defers reply

Step 3: Entering CS

- P1 receives all replies → enters CS
-

Step 4: After P1 Finishes

- P1 sends deferred REPLY to P2
-

Step 5: P2 Enters CS

- Now P2 gets all replies → enters CS
-

6. Execution Flow Summary

P1 → REQUEST → P2, P3

P2 → REQUEST → P1, P3

P2 → REPLY → P1

P1 → (defer reply to P2)

P1 enters CS

P1 exits → sends REPLY to P2

P2 enters CS

Practical 7 : Write Raymond's Tree-Based Algorithm for achieving mutual exclusion in a distributed system. Support your explanation with an example.

Concept

The **Raymond's Tree-Based Algorithm** is a **token-based algorithm** used to ensure that only one process enters the **Critical Section (CS)** at a time in a distributed system.

- Processes are arranged in a **logical tree structure**
- A unique **token** grants permission to enter CS
- Only the process holding the token can enter CS

1. Key Idea

- Each node knows its **parent (holder of token path)**
- Requests are forwarded **towards the token holder**
- Token moves along the tree to requesting process
- Each node maintains a **queue of requests**

2. Data Structures at Each Node

- **holder** → neighbor in direction of token
- **request_queue** → queue of requesting nodes
- **has_token** → boolean

3. Algorithm

When a Process Wants to Enter CS

1. If it has the token:
 - Enter CS immediately
2. Else:
 - Add itself to request queue
 - Send REQUEST to its holder
 - Wait for token

On Receiving REQUEST

1. Add requester to queue
2. If node does not have token:
 - Forward REQUEST to its holder
3. If node has token and not in CS:
 - Send token to next requester

When Receiving TOKEN

1. Set **has_token = true**
2. If it is at head of queue:
 - Enter CS
3. Else:
 - Pass token to next node in queue

On Exiting CS

1. Remove itself from queue
2. If queue not empty:
 - Send token to next requester

4. Pseudocode

Requesting Critical Section

```
function request_CS():  
    if has_token:  
        enter_CS()  
    else:  
        add self to request_queue  
  
        if not already_requested:  
            send REQUEST to holder
```

On Receiving REQUEST

```
function on_receive_request(sender):  
    add sender to request_queue  
  
    if not has_token:  
        send REQUEST to holder  
  
    else if not in_CS:  
        next = request_queue.pop()  
        send TOKEN to next  
        has_token = False
```

On Receiving TOKEN

```
function on_receive_token():  
    has_token = True  
  
    if self is in request_queue:  
        enter_CS()  
    else:  
        next = request_queue.pop()  
        send TOKEN to next  
        has_token = False
```

On Releasing CS

```
function release_CS():  
    remove self from queue  
  
    if request_queue not empty:  
        next = request_queue.pop()  
        send TOKEN to next  
        has_token = False
```

5. Detailed Example

Tree Structure

```
    P1
   / \
  P2  P3
 / \
P4  P5
```

- Initially, **P1** has the token
-

Scenario

- P5 requests CS
 - Then P4 requests CS
-

Step-by-Step Execution

Step 1: P5 Requests CS

- P5 does not have token
 - Sends REQUEST → P2
 - P2 forwards REQUEST → P1
-

Step 2: Token Moves

- P1 has token
 - Sends TOKEN → P2
 - P2 sends TOKEN → P5
-

Step 3: P5 Enters CS

- P5 receives token
 - Executes CS
-

Step 4: P4 Requests CS

- P4 sends REQUEST → P2
 - P2 forwards REQUEST toward token holder (P5)
-

Step 5: After P5 Finishes

- P5 sends TOKEN → P2
 - P2 sends TOKEN → P4
-

Step 6: P4 Enters CS

- P4 now executes CS
-

6. Message Flow Summary

P5 → P2 → P1 (REQUEST)

P1 → P2 → P5 (TOKEN)

P4 → P2 → P5 (REQUEST)

P5 → P2 → P4 (TOKEN)

Practical 8 : Write Chandy-Misra-Haas algorithm for distributed deadlock detection. Provide a detailed example to demonstrate how the algorithm detects a deadlock in a distributed system.

Concept

The **Chandy–Misra–Haas Algorithm** is used to **detect deadlocks in distributed systems** using a **probe (message-passing) approach**.

- Works on a **Wait-For Graph (WFG)**
- Detects cycles → indicates deadlock
- No central coordinator required

1. Key Idea

- When a process is blocked, it sends a **probe message**
- The probe traverses dependent processes
- If the probe returns to the initiator → **deadlock detected**

2. Probe Message Format

PROBE(i, j, k)

Where:

- **i** → initiator process
- **j** → sender
- **k** → receiver

3. Algorithm

Step 1: Initiation

1. If process **P_i** is blocked:

- For each process **P_j** it is waiting for:

send PROBE(i, i, j)

Step 2: On Receiving PROBE(i, j, k)

At process **P_k**:

1. If **P_k** is not blocked:
 - Discard probe
2. If **P_k** is blocked and waiting for other processes:
 - For each dependent process **P_m**:

send PROBE(i, k, m)
3. If **k == i**:
 - Deadlock detected

Key Features

- Fully distributed
- Uses probe messages
- Detects cycles in WFG

Advantages

- No central controller needed
- Efficient for large distributed systems
- Detects deadlock dynamically

4. Pseudocode

Initiating Process

```
function detect_deadlock(process_i):  
    if process_i is blocked:  
        for each process_j that process_i is waiting for:  
            send PROBE(i, i, j)
```

On Receiving PROBE

```
function on_receive_probe(i, j, k):  
  
    if k == i:  
        print("Deadlock detected involving process", i)  
        return  
  
    if process_k is blocked:  
        for each process_m that process_k is waiting for:  
            send PROBE(i, k, m)
```

5. Detailed Example

System Setup (Wait-For Graph)

P1 → P2

P2 → P3

P3 → P1

Step-by-Step Execution

Step 1: P1 Initiates Detection

P1 sends: PROBE(1, 1, 2)

Step 2: P2 Receives

- P2 is blocked and waiting for P3

P2 sends: PROBE(1, 2, 3)

Step 3: P3 Receives

- P3 is blocked and waiting for P1

P3 sends: PROBE(1, 3, 1)

Step 4: P1 Receives Probe Again

PROBE(1, 3, 1)

- Since receiver = initiator (P1)
 ➡ Deadlock detected
-

6. Message Flow Summary

P1 → P2 → P3 → P4

Practical 9 : Write an algorithm for dynamic load balancing using the Least Connections method. Provide a detailed example to illustrate how the algorithm works in practice.

Algorithm: Least Connections

Input:

- List of servers $S = \{S1, S2, \dots, Sn\}$
- Each server has:
 - `active_connections[i]`

Steps:

1. Initialize all servers with `active_connections = 0`.
2. When a new request arrives:
 - Check active connections of all servers.
 - Select server Si such that:

`active_connections[i]` =
minimum among all servers

3. Assign the request to server Si .
4. Increment:

`active_connections[i] += 1`

5. When a request completes:

`active_connections[i] -= 1`

6. Repeat for every incoming request.

Example

Servers: $S1 \rightarrow 3$ connections , $S2 \rightarrow 1$ connection, $S3 \rightarrow 2$ connections

Incoming Requests: R1, R2, R3, R4

Step 1: R1 arrives

- Least connections $\rightarrow S2$ (1)
- Assign $R1 \rightarrow S2$
- Updated:
 - $S1 = 3$
 - $S2 = 2$
 - $S3 = 2$

Step 2: R2 arrives

- Least connections $\rightarrow S2$ or $S3$
- Choose $S2$ (tie-breaking: first found)
- Updated:
 - $S1 = 3$
 - $S2 = 3$
 - $S3 = 2$

Step 3: R3 arrives

- Least connections $\rightarrow S3$ (2), Assign $R3 \rightarrow S3$
- Updated:
 - $S1 = 3$
 - $S2 = 3$
 - $S3 = 3$

Step 4: R4 arrives

- All equal $\rightarrow$ choose $S1$ (first)
- Updated:
 - $S1 = 4$
 - $S2 = 3$
 - $S3 = 3$

Pseudo code in Java

```
import java.util.*;

class LeastConnections {

    // Store active connections for each server
    static Map<String, Integer> servers = new HashMap<>();

    public static void main(String[] args) {
        // Initialize servers
        servers.put("S1", 0);
        servers.put("S2", 0);
        servers.put("S3", 0);

        assignRequest("R1");
        assignRequest("R2");
        assignRequest("R3");

        // Example: request on S2 completed
        completeRequest("S2");
    }

    // Find server with least connections
    static String getLeastConnectionServer() {
        String minServer = null;
        int minConnections = Integer.MAX_VALUE;

        for (String server : servers.keySet()) {
            if (servers.get(server) < minConnections) {
                minConnections = servers.get(server);
                minServer = server;
            }
        }
        return minServer;
    }

    // Assign request
    static void assignRequest(String requestId) {
        String server = getLeastConnectionServer();
        System.out.println(requestId + " assigned to " + server);

        // Increase connection count
        servers.put(server, servers.get(server) + 1);
    }

    // Complete request
    static void completeRequest(String server) {
        servers.put(server, servers.get(server) - 1);
    } }
}
```

Practical 10 : Develop an algorithm for static load balancing using the Weighted Round Robin method. Include a step-by-step example to demonstrate how tasks are distributed among servers based on their weights.

Algorithm: Weighted Round Robin

Input:

- Servers $S = \{S_1, S_2, \dots, S_n\}$
- Corresponding weights $W = \{w_1, w_2, \dots, w_n\}$

Steps:

1. Create a scheduling list where each server appears according to its weight.
 - Example:

S1 (weight 2), S2 (weight 1), S3 (weight 3)
→ Sequence: S1, S1, S2, S3, S3, S3
 2. Initialize pointer $i = 0$.
 3. For each incoming request:
 - Assign request to:

server = sequence[i]
 - Move pointer:

 $i = (i + 1) \bmod \text{length}(\text{sequence})$
 4. Repeat cyclically.
-

Example (Step-by-Step)

Servers and Weights:

- S1 → weight 2
- S2 → weight 1
- S3 → weight 3

Constructed Sequence:

[S1, S1, S2, S3, S3, S3]

Incoming Requests: R1 to R8

Request	Assigned Server
R1	S1
R2	S1
R3	S2
R4	S3
R5	S3
R6	S3
R7	S1 (cycle repeats)
R8	S1

Pseudo Code

```
import java.util.*;

class WeightedRoundRobin {

    static List<String> sequence = new ArrayList<>();
    static int index = 0;

    public static void main(String[] args) {

        // Define servers with weights
        Map<String, Integer> servers = new LinkedHashMap<>();
        servers.put("S1", 2);
        servers.put("S2", 1);
        servers.put("S3", 3);

        // Build weighted sequence
        for (String server : servers.keySet()) {
            int weight = servers.get(server);
            for (int i = 0; i < weight; i++) {
                sequence.add(server);
            }
        }

        // Assign requests
        for (int i = 1; i <= 8; i++) {
            assignRequest("R" + i);
        }
    }

    static void assignRequest(String requestId) {
        String server = sequence.get(index);
        System.out.println(requestId + " assigned to " + server);

        // Move pointer cyclically
        index = (index + 1) % sequence.size();
    }
}
```